Perfekt — hier sind die wichtigsten Erkenntnisse direkt eingebaut:

> 🔍 *Key-Findings:*
>
> - [1] [4] *Nur mit `app.isPackaged === true` den Updater ausführen — niemals in der Entwicklung*
> - [6] *electron-updater unterstützt GitHub Releases, S3, DigitalOcean, Keygen und generische HTTPS-Server — nur 2 Zeilen Code*
> - [10] *Node.js APIs wie `fs` und `https` gehören in den Main-Prozess — via IPC an React kommunizieren*
> - [12] *IPC-Renderer-Zugriff aus React-Komponenten erfordert das Preload-Muster*

🚀 Stufe 7: In-App-Download + Auto-Updater für React-Architektur

Architektur dieser Stufe (React-spezifisch)

{} text

```
┌──────────────────────────────────────────────────────────────┐
│  REACT-SPEZIFISCHE ARCHITEKTUR                               │
│                                                              │
│  electron/main.cjs       ← Downloader + Updater Logik        │
│  electron/preload.cjs    ← contextBridge Erweiterung         │
│  server/index.js         ← Express Proxy-Routen (SSE Chat-Stream)  │
│                                                              │
│  client/src/                                                 │
│    hooks/                                                    │
│      useUpdater.js       ← React Hook für Auto-Update        │
│      useDownloader.js    ← React Hook für Downloads          │
│      useServerStatus.js  ← Health-Polling Hook               │
│    pages/                                                    │
│      DownloadsPage.jsx   ← Download-Manager + Historie       │
│      ChatPage.jsx        ← Streaming Chat (VERBESSERT)       │
│    components/                                               │
│      UpdateBanner.jsx    ← Update-Benachrichtigung           │
│      DownloadBar.jsx     ← Floating Download-Fortschritt     │
│      ServerStatusBadge.jsx ← Live Health-Status              │
└──────────────────────────────────────────────────────────────┘
```

1️⃣ `electron/main.cjs` — Downloader + Updater (Ergänzung)

{} JavaScript

```
// ═══════════════════════════════════════════════════════════
//  ERGÄNZUNG zu electron/main.cjs
//  Alles UNTER den bestehenden IPC-Handlern einfügen
// ═══════════════════════════════════════════════════════════

'use strict'

// ── Neue requires ganz oben in main.cjs ergänzen ──────────
const https      = require('https')
const http       = require('http')
const fs         = require('fs')
const urlModule  = require('url')
const path       = require('path')

// electron-updater (try/catch falls noch nicht installiert)
let autoUpdater = null
try {
  autoUpdater = require('electron-updater').autoUpdater
} catch (e) {
  console.warn('[Updater] electron-updater nicht installiert:', e.message)
}

// ── Download-State ─────────────────────────────────────────
const activeDownloads = new Map()

// ═══════════════════════════════════════════════════════════
//  AUTO-UPDATER
//  KRITISCH: app.isPackaged Check — NIEMALS in Dev ausführen
// ═══════════════════════════════════════════════════════════
function initAutoUpdater(mainWindow) {
  if (!app.isPackaged) {
    console.log('[Updater] Entwicklungs-Modus — Updater deaktiviert')
    // Dev-Hinweis an Renderer
    mainWindow?.webContents.send('updater-event', {
      event: 'dev-mode',
      data:  { message: 'Auto-Updater ist nur in der gepackten App aktiv.' }
    })
    return
  }

  if (!autoUpdater) {
    console.warn('[Updater] electron-updater nicht installiert')
    return
  }

  // User entscheidet selbst — kein automatischer Download
  autoUpdater.autoDownload         = false
  autoUpdater.autoInstallOnAppQuit = true

  const sendEvent = (event, data) =>
    mainWindow?.webContents.send('updater-event', { event, data })

  autoUpdater.on('checking-for-update', () =>
    sendEvent('checking', {})
```

```
  )

  autoUpdater.on('update-available', (info) =>
    sendEvent('available', {
      version:        info.version,
      currentVersion: app.getVersion(),
      releaseDate:    info.releaseDate,
      releaseNotes:   Array.isArray(info.releaseNotes)
        ? info.releaseNotes.map(n => n.note || '').join('\n')
        : (info.releaseNotes || '')
    })
  )

  autoUpdater.on('update-not-available', () =>
    sendEvent('not-available', { currentVersion: app.getVersion() })
  )

  autoUpdater.on('download-progress', (p) =>
    sendEvent('progress', {
      percent:     Math.round(p.percent),
      transferred: p.transferred,
      total:       p.total,
      speedBps:    p.bytesPerSecond
    })
  )

  autoUpdater.on('update-downloaded', (info) =>
    sendEvent('downloaded', { version: info.version })
  )

  autoUpdater.on('error', (err) =>
    sendEvent('error', { message: err.message })
  )

  // Erster Check nach 5 Sekunden
  setTimeout(() => autoUpdater.checkForUpdates().catch(() => {}), 5000)

  // Alle 6 Stunden erneut
  setInterval(() => autoUpdater.checkForUpdates().catch(() => {}), 6 * 60 * 60 * 1000)
}

// ═══════════════════════════════════════════════════════════
//  IPC: UPDATER-HANDLER
// ═══════════════════════════════════════════════════════════
ipcMain.handle('updater:check', async () => {
  if (!app.isPackaged || !autoUpdater) {
    return {
      success:  false,
      devMode:  !app.isPackaged,
      message:  'Auto-Updater nur in gepackter App verfügbar.'
    }
  }
  try {
    await autoUpdater.checkForUpdates()
    return { success: true }
  } catch (err) {
    return { success: false, error: err.message }
```

```
  }
})

ipcMain.handle('updater:download', async () => {
  if (!app.isPackaged || !autoUpdater)
    return { success: false, error: 'Nur in gepackter App.' }
  try {
    await autoUpdater.downloadUpdate()
    return { success: true }
  } catch (err) {
    return { success: false, error: err.message }
  }
})

ipcMain.handle('updater:install', () => {
  if (!app.isPackaged || !autoUpdater)
    return { success: false, error: 'Nur in gepackter App.' }
  autoUpdater.quitAndInstall(false, true)
  return { success: true }
})

ipcMain.handle('updater:version', () => ({
  version:    app.getVersion(),
  isPackaged: app.isPackaged,
  platform:   process.platform,
  electron:   process.versions.electron,
  node:       process.versions.node
}))

// ═══════════════════════════════════════════════════════
//  IPC: IN-APP DOWNLOADER
// ═══════════════════════════════════════════════════════

ipcMain.handle('download:pickFolder', async (event) => {
  const win = BrowserWindow.fromWebContents(event.sender)
  const result = await dialog.showOpenDialog(win, {
    title:      'Zielordner für Modell-Download wählen',
    properties: ['openDirectory', 'createDirectory'],
    defaultPath: store?.get('downloadDir') || app.getPath('downloads')
  })
  if (!result.canceled && result.filePaths[0]) {
    store?.set('downloadDir', result.filePaths[0])
    return { success: true, path: result.filePaths[0] }
  }
  return { success: false }
})

ipcMain.handle('download:start', async (event, { downloadUrl, filename, targetDir }) => {
  const finalDir  = targetDir ||
                    store?.get('downloadDir') ||
                    app.getPath('downloads')
  const safeName  = filename.replace(/[^a-zA-Z0-9._\-() ]/g, '_')
  const finalPath = path.join(finalDir, safeName)

  // Ordner anlegen
  try { fs.mkdirSync(finalDir, { recursive: true }) }
  catch (e) { return { success: false, error: `Ordner-Fehler: ${e.message}` } }
```

```
  // Existenz-Check
  if (fs.existsSync(finalPath)) {
    return {
      success:    false,
      error:      'Datei existiert bereits.',
      targetPath: finalPath
    }
  }

  const dlId = `dl_${Date.now()}_${Math.random().toString(36).slice(2,8)}`
  const sender = event.sender

  // Download im Hintergrund starten
  executeDownload(dlId, downloadUrl, finalPath, sender)

  return { success: true, downloadId: dlId, targetPath: finalPath }
})

ipcMain.handle('download:cancel', (_, dlId) => {
  const dl = activeDownloads.get(dlId)
  if (dl?.cancel) { dl.cancel(); return { success: true } }
  return { success: false, error: 'Download nicht gefunden.' }
})

ipcMain.handle('download:history', () =>
  store?.get('downloadHistory', []) || []
)

ipcMain.handle('download:historyClear', () => {
  store?.set('downloadHistory', [])
  return { success: true }
})

ipcMain.handle('download:openFolder', (_, filePath) => {
  shell.showItemInFolder(filePath)
  return { success: true }
})

// ── Download-Engine ──────────────────────────────────────
function executeDownload(dlId, startUrl, targetPath, sender, maxRedirects = 5) {
  const startTs  = Date.now()
  let downloaded = 0
  let totalBytes = 0
  let cancelled  = false
  let lastEmitTs = 0
  const tempPath = targetPath + '.part'

  const emit = (event, data) => {
    // Sicher: Sender könnte zwischenzeitlich zerstört sein
    if (!sender.isDestroyed()) {
      sender.send('download:event', { downloadId: dlId, event, data })
    }
  }

  const cleanup = (ok) => {
    activeDownloads.delete(dlId)
```

```
  if (!ok) { try { fs.unlinkSync(tempPath) } catch {} }
}

function doRequest(currentUrl, redirectsLeft) {
  const parsed = urlModule.parse(currentUrl)
  const proto  = parsed.protocol === 'https:' ? https : http

  const req = proto.get({
    hostname: parsed.hostname,
    port:     parsed.port || (parsed.protocol === 'https:' ? 443 : 80),
    path:     parsed.path,
    headers:  { 'User-Agent': 'HermesControlCenter/3.0', 'Accept': '*/*' },
    timeout:  30000
  }, (res) => {

    // Redirect-Handling (HuggingFace nutzt 302)
    if (res.statusCode >= 300 && res.statusCode < 400 && res.headers.location) {
      if (redirectsLeft <= 0) {
        emit('error', { message: 'Zu viele Redirects' }); cleanup(false); return
      }
      const next = res.headers.location.startsWith('http')
        ? res.headers.location
        : `${parsed.protocol}//${parsed.hostname}${res.headers.location}`
      doRequest(next, redirectsLeft - 1)
      return
    }

    if (res.statusCode !== 200) {
      emit('error', { message: `HTTP ${res.statusCode}` }); cleanup(false); return
    }

    totalBytes = parseInt(res.headers['content-length'] || '0', 10)
    emit('started', { filename: path.basename(targetPath), totalBytes, targetPath })

    const fileStream = fs.createWriteStream(tempPath)

    res.on('data', (chunk) => {
      if (cancelled) return
      downloaded += chunk.length
      const now = Date.now()
      if (now - lastEmitTs >= 200) {  // max 5x/s
        const elapsedS = Math.max((now - startTs) / 1000, 0.01)
        const speedBps = downloaded / elapsedS
        const etaS     = totalBytes > 0
          ? (totalBytes - downloaded) / Math.max(speedBps, 1)
          : 0
        emit('progress', {
          downloaded,
          totalBytes,
          percent:    totalBytes > 0 ? Math.round((downloaded / totalBytes) * 100) : 0,
          speedBps:   Math.round(speedBps),
          etaSeconds: Math.round(etaS)
        })
        lastEmitTs = now
      }
    })
```

```
      res.pipe(fileStream)

      fileStream.on('finish', () => {
        fileStream.close(() => {
          if (cancelled) return
          try { fs.renameSync(tempPath, targetPath) }
          catch (err) {
            emit('error', { message: `Speicher-Fehler: ${err.message}` })
            cleanup(false); return
          }

          const elapsedSec = Math.round((Date.now() - startTs) / 1000)
          const entry = {
            id: dlId,
            filename:    path.basename(targetPath),
            targetPath,
            size:        downloaded,
            elapsedSec,
            downloadedAt: Date.now()
          }

          // Verlauf speichern
          const history = store?.get('downloadHistory', []) || []
          history.unshift(entry)
          if (history.length > 50) history.pop()
          store?.set('downloadHistory', history)

          emit('completed', entry)
          cleanup(true)
        })
      })

      res.on('error',     (e) => { emit('error', { message: e.message }); cleanup(false) })
      fileStream.on('error', (e) => { emit('error', { message: e.message }); cleanup(false) })
    })

    req.on('error',   (e) => { emit('error', { message: e.message }); cleanup(false) })
    req.on('timeout', ()  => {
      req.destroy()
      emit('error', { message: 'Verbindungs-Timeout' })
      cleanup(false)
    })

    activeDownloads.set(dlId, {
      cancel: () => {
        cancelled = true
        req.destroy()
        emit('cancelled', {})
        cleanup(false)
      }
    })
  }

  doRequest(startUrl, maxRedirects)
}

// ── initAutoUpdater in createWindow einbinden ──────────────────
```

```
// WICHTIG: Diese Zeile in der bestehenden createWindow()-Funktion
// nach mainWindow.once('ready-to-show', ...) einfügen:
//
//   mainWindow.once('ready-to-show', () => {
//     mainWindow.show()
//     initAutoUpdater(mainWindow)   // ← NEU
//   })
```

2 `electron/preload.cjs` — Erweitert um Updater + Downloader

{} JavaScript

```
// ═══════════════════════════════════════════════════════
//  ERGÄNZUNG zu electron/preload.cjs
//  Folgende Einträge im contextBridge.exposeInMainWorld-Objekt ergänzen:
// ═══════════════════════════════════════════════════════

// ── Auto-Updater ──────────────────────────────────────
updaterCheck:        () => ipcRenderer.invoke('updater:check'),
updaterDownload:     () => ipcRenderer.invoke('updater:download'),
updaterInstall:      () => ipcRenderer.invoke('updater:install'),
updaterVersion:      () => ipcRenderer.invoke('updater:version'),
onUpdaterEvent:     (cb) => {
  const handler = (_, data) => cb(data)
  ipcRenderer.on('updater-event', handler)
  return () => ipcRenderer.removeListener('updater-event', handler)
},

// ── Downloader ────────────────────────────────────────
downloadPickFolder:  () => ipcRenderer.invoke('download:pickFolder'),
downloadStart:      (o) => ipcRenderer.invoke('download:start', o),
downloadCancel:    (id) => ipcRenderer.invoke('download:cancel', id),
downloadHistory:     () => ipcRenderer.invoke('download:history'),
downloadHistoryClear:() => ipcRenderer.invoke('download:historyClear'),
downloadOpenFolder: (p) => ipcRenderer.invoke('download:openFolder', p),
onDownloadEvent:   (cb) => {
  const handler = (_, data) => cb(data)
  ipcRenderer.on('download:event', handler)
  return () => ipcRenderer.removeListener('download:event', handler)
},
```

3 `server/index.js` — Streaming-Route verbessert

{} JavaScript

```
// ═══════════════════════════════════════════════════════════
//  server/index.js — ERGÄNZUNGEN
//  Füge diese Routen zu deinem bestehenden Express-Server hinzu
// ═══════════════════════════════════════════════════════════

const express = require('express')
const http    = require('http')
const https   = require('https')

// ── POST /api/chat/stream (VERBESSERT) ──────────────────────
// Verbesserungen:
//  - Robusteres SSE-Parsing (llama.cpp Bug-Workaround)
//  - Timings aus letztem Chunk durchreichen
//  - Korrekte Fehlerbehandlung bei Context-Overflow
//  - AbortController-Support via Client-Disconnect
router.post('/api/chat/stream', async (req, res) => {
  const {
    messages,
    systemPrompt,
    temperature = 0.7,
    maxTokens   = 2048,
    port        = 8080
  } = req.body

  if (!messages || !Array.isArray(messages)) {
    return res.status(400).json({ error: 'messages Array fehlt.' })
  }

  // SSE-Header setzen
  res.setHeader('Content-Type',  'text/event-stream')
  res.setHeader('Cache-Control', 'no-cache')
  res.setHeader('Connection',    'keep-alive')
  res.setHeader('X-Accel-Buffering', 'no')
  res.flushHeaders()

  // API-Messages zusammenstellen
  const apiMessages = []
  if (systemPrompt?.trim()) {
    apiMessages.push({ role: 'system', content: systemPrompt })
  }
  messages.slice(-40).forEach(m =>  // Max 40 für Context-Limit
    apiMessages.push({ role: m.role, content: m.content })
  )

  const body = JSON.stringify({
    model:          'hermes',
    messages:       apiMessages,
    stream:         true,
    temperature:    parseFloat(temperature) || 0.7,
    max_tokens:     parseInt(maxTokens) || 2048,
    stream_options: { include_usage: true }
  })

  // Client-Disconnect sauber behandeln
  let clientConnected = true
  req.on('close', () => { clientConnected = false })
```

```
const sendSSE = (event, data) => {
  if (!clientConnected) return
  try {
    res.write(`event: ${event}\n`)
    res.write(`data: ${JSON.stringify(data)}\n\n`)
  } catch {}
}

// ── Request an llama-server ──────────────────────────
const options = {
  hostname: '127.0.0.1',
  port:     parseInt(port) || 8080,
  path:     '/v1/chat/completions',
  method:   'POST',
  headers:  {
    'Content-Type':   'application/json',
    'Content-Length': Buffer.byteLength(body),
    'Accept':         'text/event-stream'
  },
  timeout: 120000
}

const llamaReq = http.request(options, (llamaRes) => {
  if (llamaRes.statusCode !== 200) {
    sendSSE('error', { message: `llama-server: HTTP ${llamaRes.statusCode}` })
    res.end()
    return
  }

  let buffer = ''

  llamaRes.on('data', (chunk) => {
    if (!clientConnected) { llamaReq.destroy(); return }
    buffer += chunk.toString()

    // SSE durch \n\n getrennt
    const parts = buffer.split('\n\n')
    buffer = parts.pop() ?? ''

    for (const part of parts) {
      for (const line of part.split('\n')) {
        if (!line.startsWith('data: ')) continue
        const raw = line.slice(6).trim()
        if (raw === '[DONE]') continue

        // ── KRITISCH: Jeder Chunk einzeln try/catch ──────
        // Workaround für llama.cpp RFC-nonkonformen SSE-Bug
        let parsed
        try {
          parsed = JSON.parse(raw)
        } catch {
          // Nicht-JSON-Payload → als Fehler-Hinweis senden
          if (raw.length > 0) {
            sendSSE('warning', { message: `Non-JSON SSE: ${raw.slice(0, 80)}` })
          }
          continue
```

```
        }

        // ── OAI-Format ──────────────────────────────────
        const delta = parsed.choices?.[0]?.delta?.content
        if (delta !== undefined && delta !== null) {
          sendSSE('token', { content: delta })
        }

        // ── llama.cpp non-OAI Fallback ──────────────────
        if (parsed.content !== undefined && delta === undefined) {
          sendSSE('token', { content: parsed.content })
        }

        // ── Usage-Daten (letzter Chunk) ─────────────────
        // llama.cpp sendet usage + timings zusammen mit stop
        if (parsed.usage) {
          sendSSE('usage', {
            promptTokens:     parsed.usage.prompt_tokens     || 0,
            completionTokens: parsed.usage.completion_tokens || 0,
            totalTokens:      parsed.usage.total_tokens      || 0
          })
        }

        // ── llama.cpp Timings ───────────────────────────
        if (parsed.timings) {
          sendSSE('timings', {
            tokensPerSecond:  parsed.timings.predicted_per_second || 0,
            promptMs:         parsed.timings.prompt_ms            || 0,
            predictedMs:      parsed.timings.predicted_ms         || 0
          })
        }

        // Stop-Signal
        if (
          parsed.choices?.[0]?.finish_reason === 'stop' ||
          parsed.stop === true
        ) {
          sendSSE('stop', { finishReason: 'stop' })
        }
      }
    }
  })

  llamaRes.on('end', () => {
    sendSSE('done', {})
    res.end()
  })

  llamaRes.on('error', (err) => {
    sendSSE('error', { message: err.message })
    res.end()
  })
})

llamaReq.on('error', (err) => {
  sendSSE('error', {
    message: `llama-server nicht erreichbar: ${err.message}`,
```

```
      hint:    'Stelle sicher, dass der llama-server läuft.'
    })
    res.end()
  })

  llamaReq.on('timeout', () => {
    llamaReq.destroy()
    sendSSE('error', { message: 'Timeout — llama-server antwortet nicht.' })
    res.end()
  })

  llamaReq.write(body)
  llamaReq.end()
})

// ── GET /api/health ──────────────────────────────────────────
router.get('/api/health', async (req, res) => {
  const port = parseInt(req.query.port) || 8080
  const check = () => new Promise((resolve) => {
    const r = http.get(
      `http://127.0.0.1:${port}/health`,
      { timeout: 3000 },
      (resp) => {
        let data = ''
        resp.on('data', d => data += d)
        resp.on('end', () => {
          try   { resolve({ ok: true, ...JSON.parse(data) }) }
          catch { resolve({ ok: true, status: 'ok' }) }
        })
      }
    )
    r.on('error', () => resolve({ ok: false, status: 'unreachable' }))
    r.on('timeout', () => { r.destroy(); resolve({ ok: false, status: 'timeout' }) })
  })

  const result = await check()
  res.json(result)
})

// ── GET /api/models ──────────────────────────────────────────
router.get('/api/models', async (req, res) => {
  const port = parseInt(req.query.port) || 8080
  const r = http.get(
    `http://127.0.0.1:${port}/v1/models`,
    { timeout: 3000 },
    (resp) => {
      let data = ''
      resp.on('data', d => data += d)
      resp.on('end', () => {
        try   { res.json(JSON.parse(data)) }
        catch { res.json({ data: [] }) }
      })
    }
  )
  r.on('error', () => res.json({ data: [], error: 'Nicht erreichbar' }))
})
```

client/src/hooks/useUpdater.js

{} JavaScript

```
// ═══════════════════════════════════════════════════════
//  useUpdater.js — React Hook für Auto-Updater
// ═══════════════════════════════════════════════════════

import { useState, useEffect, useCallback } from 'react'

const isElectron = () => Boolean(window.electronAPI)

export function useUpdater() {
  const [status,   setStatus]   = useState('idle')
  const [info,     setInfo]     = useState(null)
  const [progress, setProgress] = useState(null)
  const [version,  setVersion]  = useState(null)
  const [isDevMode,setIsDevMode]= useState(false)

  // App-Version laden
  useEffect(() => {
    if (!isElectron()) return
    window.electronAPI.updaterVersion().then(v => {
      setVersion(v)
      setIsDevMode(!v.isPackaged)
    }).catch(() => {})
  }, [])

  // Updater-Events registrieren
  useEffect(() => {
    if (!isElectron()) return

    const cleanup = window.electronAPI.onUpdaterEvent(({ event, data }) => {
      switch (event) {
        case 'dev-mode':
          setIsDevMode(true)
          break
        case 'checking':
          setStatus('checking')
          break
        case 'available':
          setStatus('available')
          setInfo(data)
          break
        case 'not-available':
          setStatus('idle')
          break
        case 'progress':
          setStatus('downloading')
          setProgress(data)
```

```
          break
        case 'downloaded':
          setStatus('downloaded')
          setInfo(prev => ({ ...prev, ...data }))
          setProgress(null)
          break
        case 'error':
          setStatus('error')
          setInfo({ error: data.message })
          break
        default:
          break
      }
    })

    // Cleanup-Funktion aus onUpdaterEvent
    return () => { if (typeof cleanup === 'function') cleanup() }
  }, [])

  const checkForUpdates = useCallback(async () => {
    if (!isElectron()) return { success: false, error: 'Kein Electron' }
    setStatus('checking')
    const r = await window.electronAPI.updaterCheck()
    if (!r.success) setStatus('idle')
    return r
  }, [])

  const downloadUpdate = useCallback(async () => {
    if (!isElectron()) return
    setStatus('downloading')
    await window.electronAPI.updaterDownload()
  }, [])

  const installUpdate = useCallback(async () => {
    if (!isElectron()) return
    await window.electronAPI.updaterInstall()
  }, [])

  return {
    status,
    info,
    progress,
    version,
    isDevMode,
    checkForUpdates,
    downloadUpdate,
    installUpdate
  }
}
```

5 client/src/hooks/useDownloader.js

{} JavaScript

```
// ═══════════════════════════════════════════════════════════
//  useDownloader.js — React Hook für In-App Downloads
// ═══════════════════════════════════════════════════════════

import { useState, useEffect, useCallback, useRef } from 'react'

const isElectron = () => Boolean(window.electronAPI)

export function useDownloader() {
  // Map<downloadId, DownloadState>
  const [downloads, setDownloads] = useState(new Map())
  const downloadsRef = useRef(downloads)
  downloadsRef.current = downloads

  // Download-Events registrieren
  useEffect(() => {
    if (!isElectron()) return

    const cleanup = window.electronAPI.onDownloadEvent(({ downloadId, event, data }) => {
      setDownloads(prev => {
        const next = new Map(prev)

        switch (event) {
          case 'started':
            next.set(downloadId, {
              id:         downloadId,
              filename:   data.filename,
              totalBytes: data.totalBytes,
              targetPath: data.targetPath,
              status:     'downloading',
              percent:    0,
              speedBps:   0,
              etaSeconds: 0,
              downloaded: 0,
              startedAt:  Date.now()
            })
            break

          case 'progress':
            if (next.has(downloadId)) {
              next.set(downloadId, {
                ...next.get(downloadId),
                ...data,
                status: 'downloading'
              })
            }
            break

          case 'completed':
            next.set(downloadId, {
              ...(next.get(downloadId) || {}),
              ...data,
              status:    'completed',
              percent:   100,
```

```
            completedAt: Date.now()
          })
          // Nach 5s aus der aktiven Liste entfernen
          setTimeout(() => {
            setDownloads(p => {
              const m = new Map(p)
              m.delete(downloadId)
              return m
            })
          }, 5000)
          break

        case 'error':
          next.set(downloadId, {
            ...(next.get(downloadId) || {}),
            status:  'error',
            error:   data.message,
            errorAt: Date.now()
          })
          break

        case 'cancelled':
          next.delete(downloadId)
          break

        default:
          break
      }

      return next
    })
  })

  return () => { if (typeof cleanup === 'function') cleanup() }
}, [])

// ── Öffentliche API ──────────────────────────────────────
const startDownload = useCallback(async ({ downloadUrl, filename, targetDir }) => {
  if (!isElectron()) return { success: false }

  const result = await window.electronAPI.downloadStart({
    downloadUrl,
    filename,
    targetDir
  })

  return result
}, [])

const pickFolder = useCallback(async () => {
  if (!isElectron()) return { success: false }
  return window.electronAPI.downloadPickFolder()
}, [])

const cancelDownload = useCallback(async (id) => {
  if (!isElectron()) return
  await window.electronAPI.downloadCancel(id)
```

```
  }, [])

  const openFolder = useCallback((filePath) => {
    if (!isElectron()) return
    window.electronAPI.downloadOpenFolder(filePath)
  }, [])

  const getHistory = useCallback(async () => {
    if (!isElectron()) return []
    return window.electronAPI.downloadHistory()
  }, [])

  const clearHistory = useCallback(async () => {
    if (!isElectron()) return
    await window.electronAPI.downloadHistoryClear()
  }, [])

  // Konvertiert Map zu Array für einfaches Rendern
  const activeDownloads = Array.from(downloads.values())
    .filter(d => d.status === 'downloading')

  const completedRecent = Array.from(downloads.values())
    .filter(d => d.status === 'completed')

  return {
    downloads,
    activeDownloads,
    completedRecent,
    startDownload,
    pickFolder,
    cancelDownload,
    openFolder,
    getHistory,
    clearHistory
  }
}
```

6 client/src/hooks/useServerStatus.js

{} JavaScript

```
// ═══════════════════════════════════════════════════════
//  useServerStatus.js — Polling Hook für llama-server Health
// ═══════════════════════════════════════════════════════

import { useState, useEffect, useRef, useCallback } from 'react'

export function useServerStatus(port = 8080, intervalMs = 5000) {
  const [status,    setStatus]    = useState('unknown') // unknown | ok | loading | error
  const [lastCheck, setLastCheck]= useState(null)
```

```
  const [modelName, setModelName]= useState(null)
  const intervalRef = useRef(null)
  const portRef     = useRef(port)
  portRef.current = port

  const checkHealth = useCallback(async () => {
    try {
      const resp = await fetch(
        `/api/health?port=${portRef.current}`,
        { signal: AbortSignal.timeout(4000) }
      )
      if (!resp.ok) throw new Error(`HTTP ${resp.status}`)
      const data = await resp.json()
      setStatus(data.status === 'loading model' ? 'loading' : 'ok')
      setLastCheck(Date.now())
    } catch {
      setStatus('error')
      setLastCheck(Date.now())
    }
  }, [])

  const loadModelName = useCallback(async () => {
    try {
      const resp = await fetch(`/api/models?port=${portRef.current}`,
        { signal: AbortSignal.timeout(3000) })
      const data = await resp.json()
      const name = data.data?.[0]?.id
      if (name) setModelName(name)
    } catch {}
  }, [])

  useEffect(() => {
    checkHealth()
    loadModelName()

    intervalRef.current = setInterval(() => {
      checkHealth()
    }, intervalMs)

    return () => {
      if (intervalRef.current) clearInterval(intervalRef.current)
    }
  }, [checkHealth, loadModelName, intervalMs])

  const refreshNow = useCallback(() => {
    checkHealth()
    loadModelName()
  }, [checkHealth, loadModelName])

  return { status, lastCheck, modelName, refreshNow }
}
```

7 client/src/components/UpdateBanner.jsx

{} React

```jsx
// ═══════════════════════════════════════════════════════
//  UpdateBanner.jsx — Update-Benachrichtigung
// ═══════════════════════════════════════════════════════

import React from 'react'
import { useUpdater } from '../hooks/useUpdater'

export function UpdateBanner() {
  const {
    status, info, progress,
    downloadUpdate, installUpdate
  } = useUpdater()

  // Kein Banner bei idle oder checking
  if (status === 'idle' || status === 'checking') return null

  return (
    <div className={`update-banner ub-${status}`} role="alert">
      <div className="ub-content">

        {/* Update verfügbar */}
        {status === 'available' && (
          <>
            <div className="ub-texts">
              <span className="ub-title">
                📦 Update v{info?.version} verfügbar
              </span>
              <span className="ub-sub">
                Aktuell: v{info?.currentVersion}
              </span>
            </div>
            <div className="ub-actions">
              <button
                className="btn btn-primary btn-sm"
                onClick={downloadUpdate}
              >
                ⬇️ Herunterladen
              </button>
              <button
                className="btn btn-ghost btn-sm"
                onClick={() => alert(info?.releaseNotes || 'Keine Details.')}
              >
                📋 Details
              </button>
            </div>
          </>
        )}

        {/* Download läuft */}
        {status === 'downloading' && progress && (
          <>
```

```
            <div className="ub-texts">
              <span className="ub-title">
                ⬇️ Update lädt… {progress.percent}%
              </span>
              <span className="ub-sub">
                {formatBytes(progress.transferred)} /
                {formatBytes(progress.total)} ·
                {formatBytes(progress.speedBps)}/s
              </span>
            </div>
            <div className="ub-progress-wrap">
              <div className="ub-progress-bar">
                <div
                  className="ub-progress-fill"
                  style={{ width: `${progress.percent}%` }}
                />
              </div>
            </div>
          </>
        )}

        {/* Bereit zum Installieren */}
        {status === 'downloaded' && (
          <>
            <div className="ub-texts">
              <span className="ub-title">
                ✅ v{info?.version} bereit
              </span>
              <span className="ub-sub">
                App wird zum Installieren neu gestartet
              </span>
            </div>
            <div className="ub-actions">
              <button
                className="btn btn-success btn-sm"
                onClick={installUpdate}
              >
                🚀 Installieren & Neustart
              </button>
            </div>
          </>
        )}

        {/* Fehler */}
        {status === 'error' && (
          <div className="ub-texts">
            <span className="ub-title">⚠️ Update-Fehler</span>
            <span className="ub-sub">{info?.error}</span>
          </div>
        )}

      </div>
    </div>
  )
}

function formatBytes(bytes) {
```

```
  if (!bytes) return '0 B'
  if (bytes < 1024)       return `${bytes} B`
  if (bytes < 1048576)    return `${(bytes/1024).toFixed(1)} KB`
  if (bytes < 1073741824) return `${(bytes/1048576).toFixed(1)} MB`
  return `${(bytes/1073741824).toFixed(2)} GB`
}
```

8 client/src/components/DownloadBar.jsx

{} React

```
// ═══════════════════════════════════════════════════
//  DownloadBar.jsx — Floating Download-Fortschritt
// ═══════════════════════════════════════════════════

import React, { useState } from 'react'

export function DownloadBar({ activeDownloads, onCancel }) {
  const [minimized, setMinimized] = useState(false)

  if (!activeDownloads || activeDownloads.length === 0) return null

  return (
    <div className="dl-float-bar">
      <div className="dlb-header">
        <span className="dlb-title">
          ⬇️ Downloads ({activeDownloads.length})
        </span>
        <button
          className="btn btn-ghost btn-xs"
          onClick={() => setMinimized(m => !m)}
          title={minimized ? 'Aufklappen' : 'Minimieren'}
        >
          {minimized ? '▲' : '–'}
        </button>
      </div>

      {!minimized && (
        <div className="dlb-list">
          {activeDownloads.map(dl => (
            <DownloadItem
              key={dl.id}
              dl={dl}
              onCancel={() => onCancel(dl.id)}
            />
          ))}
        </div>
      )}
    </div>
  )
```

```
}

function DownloadItem({ dl, onCancel }) {
  const mb    = ((dl.downloaded || 0) / 1048576).toFixed(1)
  const mbTot = ((dl.totalBytes || 0) / 1048576).toFixed(1)
  const mbSec = ((dl.speedBps  || 0) / 1048576).toFixed(2)
  const eta   = formatEta(dl.etaSeconds)
  const pct   = dl.percent || 0

  return (
    <div className="dlb-item">
      <div className="dlb-row">
        <span className="dlb-name" title={dl.filename}>
          📄 {dl.filename || 'Lade…'}
        </span>
        <button
          className="btn btn-ghost btn-xs dlb-cancel"
          onClick={onCancel}
          title="Abbrechen"
        >
          ✕
        </button>
      </div>

      <div className="dlb-stats">
        <span>{mb} / {mbTot} MB</span>
        <span>{mbSec} MB/s</span>
        <span>ETA {eta}</span>
        <span className="dlb-pct">{pct}%</span>
      </div>

      <div className="dlb-bar-wrap">
        <div className="dlb-bar">
          <div className="dlb-fill" style={{ width: `${pct}%` }} />
        </div>
      </div>

      {dl.status === 'completed' && (
        <div className="dlb-done">✅ Abgeschlossen</div>
      )}
      {dl.status === 'error' && (
        <div className="dlb-error">❌ {dl.error}</div>
      )}
    </div>
  )
}

function formatEta(s) {
  if (!s || !isFinite(s) || s <= 0) return '–'
  if (s < 60)   return `${Math.round(s)}s`
  if (s < 3600) return `${Math.floor(s/60)}m ${Math.round(s%60)}s`
  return `${Math.floor(s/3600)}h ${Math.floor((s%3600)/60)}m`
}
```

client/src/pages/DownloadsPage.jsx

{} React

```jsx
// ═══════════════════════════════════════════════════════════
//  DownloadsPage.jsx — Download-Manager + Verlauf
// ═══════════════════════════════════════════════════════════

import React, { useState, useEffect, useCallback } from 'react'
import { useDownloader } from '../hooks/useDownloader'

export function DownloadsPage() {
  const { activeDownloads, cancelDownload, getHistory,
          clearHistory, openFolder } = useDownloader()

  const [history,  setHistory]  = useState([])
  const [loading,  setLoading]  = useState(true)
  const [tab,      setTab]      = useState('active') // active | history

  const loadHistory = useCallback(async () => {
    setLoading(true)
    const h = await getHistory()
    setHistory(h || [])
    setLoading(false)
  }, [getHistory])

  useEffect(() => {
    loadHistory()
  }, [loadHistory])

  const handleClearHistory = async () => {
    if (!window.confirm('Download-Verlauf wirklich leeren?')) return
    await clearHistory()
    setHistory([])
  }

  return (
    <div className="page-content">
      <div className="page-header">
        <h1>⬇️ Downloads</h1>
        <p className="page-subtitle">
          GGUF-Modelle direkt in die App laden
        </p>
      </div>

      {/* Tabs */}
      <div className="tab-bar">
        <button
          className={`tab-btn ${tab === 'active' ? 'active' : ''}`}
          onClick={() => setTab('active')}
        >
          Aktive Downloads
          {activeDownloads.length > 0 && (
            <span className="tab-badge">{activeDownloads.length}</span>
          )}
```

```
          </button>
          <button
            className={`tab-btn ${tab === 'history' ? 'active' : ''}`}
            onClick={() => { setTab('history'); loadHistory() }}
          >
            Verlauf
            {history.length > 0 && (
              <span className="tab-count">{history.length}</span>
            )}
          </button>
        </div>

        {/* Aktive Downloads */}
        {tab === 'active' && (
          <div className="tab-content">
            {activeDownloads.length === 0 ? (
              <div className="empty-state">
                <div className="es-icon">⬇️</div>
                <h3>Keine aktiven Downloads</h3>
                <p>
                  Starte einen Download über den{' '}
                  <strong>Modell-Browser</strong>.
                </p>
              </div>
            ) : (
              <div className="dl-active-list">
                {activeDownloads.map(dl => (
                  <ActiveDownloadCard
                    key={dl.id}
                    dl={dl}
                    onCancel={() => cancelDownload(dl.id)}
                  />
                ))}
              </div>
            )}
          </div>
        )}

        {/* Verlauf */}
        {tab === 'history' && (
          <div className="tab-content">
            {history.length > 0 && (
              <div className="dl-history-toolbar">
                <button
                  className="btn btn-ghost btn-sm"
                  onClick={handleClearHistory}
                >
                  🗑 Verlauf leeren
                </button>
                <span className="dl-count">
                  {history.length} Einträge
                </span>
              </div>
            )}

            {loading ? (
              <div className="loading-state">⏳ Lade Verlauf…</div>
```

```
          ) : history.length === 0 ? (
            <div className="empty-state">
              <div className="es-icon">📋</div>
              <h3>Kein Download-Verlauf</h3>
              <p>Abgeschlossene Downloads erscheinen hier.</p>
            </div>
          ) : (
            <div className="dl-history-list">
              {history.map(item => (
                <HistoryCard
                  key={item.id}
                  item={item}
                  onOpenFolder={() => openFolder(item.targetPath)}
                />
              ))}
            </div>
          )}
        </div>
      )}
    </div>
  )
}

// ── Aktiver Download ──────────────────────────────────────
function ActiveDownloadCard({ dl, onCancel }) {
  const pct   = dl.percent || 0
  const mb    = ((dl.downloaded || 0) / 1048576).toFixed(1)
  const mbTot = ((dl.totalBytes || 0) / 1048576).toFixed(1)
  const mbSec = ((dl.speedBps  || 0) / 1048576).toFixed(2)
  const eta   = formatEta(dl.etaSeconds)

  return (
    <div className="dl-card">
      <div className="dlc-header">
        <div className="dlc-icon">📦</div>
        <div className="dlc-info">
          <div className="dlc-name">{dl.filename}</div>
          <div className="dlc-stats">
            <span>{mb} / {mbTot} MB</span>
            <span>⚡ {mbSec} MB/s</span>
            <span>ETA {eta}</span>
          </div>
        </div>
        <button
          className="btn btn-danger btn-sm"
          onClick={onCancel}
        >
          ✕ Abbrechen
        </button>
      </div>

      <div className="dlc-progress">
        <div className="dlc-bar">
          <div
            className="dlc-fill"
            style={{ width: `${pct}%` }}
          />
```

```
        </div>
        <span className="dlc-pct">{pct}%</span>
      </div>
    </div>
  )
}

// ── Verlaufs-Eintrag ──────────────────────────────────────
function HistoryCard({ item, onOpenFolder }) {
  const mb   = ((item.size || 0) / 1048576).toFixed(1)
  const avgS = item.size && item.elapsedSec
    ? ((item.size / item.elapsedSec) / 1048576).toFixed(1)
    : '?'
  const date = new Date(item.downloadedAt).toLocaleString('de-DE')

  return (
    <div className="dl-history-card">
      <div className="dhc-icon">✅</div>
      <div className="dhc-body">
        <div className="dhc-name">{item.filename}</div>
        <div className="dhc-meta">
          <span>🗓️ {date}</span>
          <span>💾 {mb} MB</span>
          <span>⚡ ⌀ {avgS} MB/s</span>
          <span>⏱️ {item.elapsedSec}s</span>
        </div>
        <div className="dhc-path" title={item.targetPath}>
          {item.targetPath}
        </div>
      </div>
      <div className="dhc-actions">
        <button
          className="btn btn-ghost btn-sm"
          onClick={onOpenFolder}
        >
          📂 Ordner
        </button>
      </div>
    </div>
  )
}

function formatEta(s) {
  if (!s || !isFinite(s) || s <= 0) return '–'
  if (s < 60)   return `${Math.round(s)}s`
  if (s < 3600) return `${Math.floor(s/60)}m ${Math.round(s%60)}s`
  return `${Math.floor(s/3600)}h ${Math.floor((s%3600)/60)}m`
}
```

10 `client/src/pages/ChatPage.jsx` — Verbessert mit Streaming-Fixes

{} React

```jsx
// ═══════════════════════════════════════════════════════════
//  ChatPage.jsx — VERBESSERT
//  - Robustes SSE-Event-Parsing (structured events)
//  - Timings aus Backend durchreichen
//  - Multi-Konversation
//  - Export .md + .json
//  - Persistenz via Electron oder localStorage
// ═══════════════════════════════════════════════════════════

import React, {
  useState, useEffect, useRef,
  useCallback, useMemo
} from 'react'
import { useServerStatus } from '../hooks/useServerStatus'

const isElectron = () => Boolean(window.electronAPI)

// ── Hilfs-Hook: Konversations-Persistenz ──────────────────
function useConversations() {
  const [convs,       setConvs]       = useState([])
  const [activeId,    setActiveId]    = useState(null)

  useEffect(() => {
    loadConvs()
  }, [])

  async function loadConvs() {
    try {
      let saved = []
      if (isElectron()) {
        saved = await window.electronAPI.conversationsGet?.() || []
      } else {
        const raw = localStorage.getItem('hermes_convs')
        saved = raw ? JSON.parse(raw) : []
      }
      setConvs(saved)
      if (saved.length > 0) setActiveId(saved[0].id)
    } catch {}
  }

  async function saveConvs(list) {
    try {
      if (isElectron()) {
        await window.electronAPI.conversationsSave?.(list)
      } else {
        localStorage.setItem('hermes_convs', JSON.stringify(list.slice(0, 20)))
      }
    } catch {}
  }

  function newConv() {
    const conv = {
      id:        `conv_${Date.now()}`,
      title:     'Neue Unterhaltung',
```

```
      messages:  [],
      createdAt: Date.now(),
      updatedAt: Date.now()
    }
    const next = [conv, ...convs]
    setConvs(next)
    setActiveId(conv.id)
    saveConvs(next)
    return conv
  }

  function updateConv(id, updater) {
    setConvs(prev => {
      const next = prev.map(c => c.id === id ? updater(c) : c)
      saveConvs(next)
      return next
    })
  }

  function deleteConv(id) {
    setConvs(prev => {
      const next = prev.filter(c => c.id !== id)
      if (activeId === id) setActiveId(next[0]?.id || null)
      saveConvs(next)
      return next
    })
    if (isElectron()) {
      window.electronAPI.conversationsDelete?.(id)
    }
  }

  const activeConv = convs.find(c => c.id === activeId) || null

  return {
    convs, activeId, activeConv,
    setActiveId, newConv, updateConv, deleteConv
  }
}

// ── Haupt-Komponente ──────────────────────────────────────
export default function ChatPage({ serverPort = 8080 }) {
  const {
    convs, activeId, activeConv,
    setActiveId, newConv, updateConv, deleteConv
  } = useConversations()

  const { status: serverStatus, modelName } = useServerStatus(serverPort)

  const [input,       setInput]       = useState('')
  const [streaming,   setStreaming]   = useState(false)
  const [systemPrompt,setSystemPrompt]=useState(
    'Du bist Hermes, ein hilfreicher, ehrlicher KI-Assistent.'
  )
  const [temperature, setTemperature]= useState(0.7)
  const [maxTokens,   setMaxTokens]  = useState(2048)
  const [stats,       setStats]      = useState(null)
```

```
  const abortRef    = useRef(null)
  const messagesRef = useRef(null)

  // Auto-Scroll
  useEffect(() => {
    if (messagesRef.current) {
      messagesRef.current.scrollTop = messagesRef.current.scrollHeight
    }
  }, [activeConv?.messages])

  // ── Senden ──────────────────────────────────────────
  const sendMessage = useCallback(async () => {
    const text = input.trim()
    if (!text || streaming) return

    let conv = activeConv
    if (!conv) conv = newConv()

    const userMsg = { role: 'user', content: text, ts: Date.now() }
    const asstMsg = { role: 'assistant', content: '', ts: Date.now(), streaming: true }

    updateConv(conv.id, c => ({
      ...c,
      title: c.messages.length === 0 ? text.slice(0, 40) : c.title,
      messages: [...c.messages, userMsg, asstMsg],
      updatedAt: Date.now()
    }))

    setInput('')
    setStreaming(true)
    setStats(null)

    const controller = new AbortController()
    abortRef.current = controller

    const startTs    = Date.now()
    let tokenCount   = 0
    let firstTokenTs = 0
    let usageData    = null
    let timingsData  = null
    let accumulated  = ''

    try {
      const resp = await fetch('/api/chat/stream', {
        method:  'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          messages:     [...(conv.messages || []), userMsg],
          systemPrompt,
          temperature,
          maxTokens,
          port: serverPort
        }),
        signal: controller.signal
      })

      if (!resp.ok) throw new Error(`HTTP ${resp.status}`)
```

```
const reader  = resp.body.getReader()
const decoder = new TextDecoder()
let   buffer  = ''

while (true) {
  const { done, value } = await reader.read()
  if (done) break

  buffer += decoder.decode(value, { stream: true })
  const parts = buffer.split('\n\n')
  buffer = parts.pop() ?? ''

  for (const part of parts) {
    let eventType = 'message'
    let eventData = ''

    for (const line of part.split('\n')) {
      if (line.startsWith('event: '))      eventType = line.slice(7)
      else if (line.startsWith('data: '))  eventData = line.slice(6)
    }

    if (!eventData) continue

    let parsed
    try   { parsed = JSON.parse(eventData) }
    catch { continue }

    switch (eventType) {
      case 'token':
        if (!firstTokenTs) firstTokenTs = Date.now()
        tokenCount++
        accumulated += parsed.content || ''

        // Update im State
        updateConv(conv.id, c => {
          const msgs = [...c.messages]
          const last = { ...msgs[msgs.length - 1] }
          last.content = accumulated
          msgs[msgs.length - 1] = last
          return { ...c, messages: msgs }
        })
        break

      case 'usage':
        usageData = parsed
        break

      case 'timings':
        timingsData = parsed
        break

      case 'stop':
        break

      case 'error':
        accumulated += `\n\n❌ **Fehler:** ${parsed.message}`
```

```
            updateConv(conv.id, c => {
              const msgs = [...c.messages]
              const last = { ...msgs[msgs.length - 1] }
              last.content = accumulated
              last.error   = true
              msgs[msgs.length - 1] = last
              return { ...c, messages: msgs }
            })
            break

          case 'warning':
            console.warn('[Chat] Server-Warnung:', parsed.message)
            break

          default:
            break
        }
      }
    }

  } catch (err) {
    if (err.name !== 'AbortError') {
      const errMsg = `❌ **Verbindungsfehler:** ${err.message}`
      updateConv(conv.id, c => {
        const msgs = [...c.messages]
        const last = { ...msgs[msgs.length - 1] }
        last.content = errMsg
        last.error   = true
        msgs[msgs.length - 1] = last
        return { ...c, messages: msgs }
      })
    }
  } finally {
    const elapsed = Date.now() - startTs
    const tps     = firstTokenTs
      ? (tokenCount / Math.max((Date.now() - firstTokenTs) / 1000, 0.01)).toFixed(1)
      : '0'
    const ttft    = firstTokenTs
      ? ((firstTokenTs - startTs) / 1000).toFixed(2)
      : '?'

    const finalStats = {
      tokens:   tokenCount,
      tps,
      ttftSec:  ttft,
      elapsedMs: elapsed,
      promptTokens:     usageData?.promptTokens     || 0,
      completionTokens: usageData?.completionTokens || 0,
      tokensPerSecServer: timingsData?.tokensPerSecond
    }

    setStats(finalStats)

    // Streaming-Flag entfernen + finale Stats hinzufügen
    updateConv(conv.id, c => {
      const msgs = [...c.messages]
      const last = { ...msgs[msgs.length - 1] }
```

```
        last.streaming = false
        last.stats     = finalStats
        msgs[msgs.length - 1] = last
        return { ...c, messages: msgs, updatedAt: Date.now() }
      })

      setStreaming(false)
      abortRef.current = null
    }
  }, [
    input, streaming, activeConv, newConv, updateConv,
    systemPrompt, temperature, maxTokens, serverPort
  ])

  const stopStreaming = useCallback(() => {
    abortRef.current?.abort()
  }, [])

  // ── Export ──────────────────────────────────────────────
  const exportChat = useCallback((format) => {
    if (!activeConv?.messages?.length) return

    let content, ext
    if (format === 'md') {
      const lines = [
        `# ${activeConv.title}`,
        `_Export: ${new Date().toLocaleString('de-DE')}_`,
        '',
        ...activeConv.messages.map(m => [
          `### ${m.role === 'user' ? '👤 Du' : '🧠 Hermes'}`,
          m.content,
          m.stats ? `_${m.stats.tokens} tok · ${m.stats.tps} tok/s_` : '',
          ''
        ].join('\n'))
      ]
      content = lines.join('\n')
      ext = 'md'
    } else {
      content = JSON.stringify({
        title:     activeConv.title,
        createdAt: activeConv.createdAt,
        messages:  activeConv.messages,
        systemPrompt
      }, null, 2)
      ext = 'json'
    }

    const blob = new Blob([content], { type: 'text/plain' })
    const url  = URL.createObjectURL(blob)
    const a    = Object.assign(document.createElement('a'), {
      href: url, download: `hermes-chat-${Date.now()}.${ext}`
    })
    a.click()
    URL.revokeObjectURL(url)
  }, [activeConv, systemPrompt])

  // ── Render ──────────────────────────────────────────────
```

```
  return (
    <div className="chat-root">

      {/* ── KONVERSATIONS-SIDEBAR ──────────────────────────── */}
      <aside className="conv-sidebar">
        <div className="conv-top">
          <button
            className="btn btn-primary btn-sm"
            onClick={newConv}
            disabled={streaming}
          >
            + Neu
          </button>
        </div>

        <div className="conv-list">
          {convs.length === 0 ? (
            <div className="conv-empty">Noch keine Gespräche</div>
          ) : convs.map(c => (
            <div
              key={c.id}
              className={`conv-item ${c.id === activeId ? 'active' : ''}`}
              onClick={() => !streaming && setActiveId(c.id)}
            >
              <div className="ci-body">
                <div className="ci-title">{c.title}</div>
                <div className="ci-meta">
                  {c.messages.length} Nachr. · {relTime(c.updatedAt)}
                </div>
              </div>
              <button
                className="ci-del"
                onClick={(e) => {
                  e.stopPropagation()
                  if (window.confirm('Gespräch löschen?')) deleteConv(c.id)
                }}
              >
                ✕
              </button>
            </div>
          ))}
        </div>
      </aside>

      {/* ── CHAT-AREA ──────────────────────────────────────── */}
      <div className="chat-main">

        {/* Header */}
        <div className="chat-header">
          <div className="ch-left">
            <ServerStatusBadge status={serverStatus} />
            {modelName && (
              <span className="ch-model">🧠 {modelName}</span>
            )}
          </div>
          <div className="ch-right">
            <button
```

```
            className="btn btn-ghost btn-sm"
            onClick={() => exportChat('md')}
            disabled={!activeConv?.messages?.length}
          >
            📄 .md
          </button>
          <button
            className="btn btn-ghost btn-sm"
            onClick={() => exportChat('json')}
            disabled={!activeConv?.messages?.length}
          >
            📋 .json
          </button>
        </div>
      </div>

      {/* Optionen (eingeklappt) */}
      <details className="chat-options">
        <summary>⚙️ Optionen</summary>
        <div className="co-grid">
          <div className="co-field">
            <label>System-Prompt</label>
            <textarea
              value={systemPrompt}
              onChange={e => setSystemPrompt(e.target.value)}
              rows={2}
              disabled={streaming}
            />
          </div>
          <div className="co-row">
            <div className="co-field">
              <label>Temp: {temperature.toFixed(2)}</label>
              <input
                type="range" min="0" max="2" step="0.05"
                value={temperature}
                onChange={e => setTemperature(parseFloat(e.target.value))}
              />
            </div>
            <div className="co-field">
              <label>Max Tokens</label>
              <input
                type="number" value={maxTokens} min="64" max="16384"
                onChange={e => setMaxTokens(parseInt(e.target.value) || 2048)}
              />
            </div>
          </div>
        </div>
      </details>

      {/* Nachrichten */}
      <div className="chat-messages" ref={messagesRef}>
        {!activeConv || activeConv.messages.length === 0 ? (
          <EmptyState
            serverStatus={serverStatus}
            onSuggestion={q => setInput(q)}
          />
        ) : (
```

```
            activeConv.messages.map((msg, i) => (
              <MessageBubble key={i} msg={msg} />
            ))
          )}
        </div>

        {/* Stats */}
        {stats && !streaming && (
          <div className="chat-stats-bar">
            ⚡ {stats.tokens} tok ·
            {stats.tps} tok/s ·
            TTFT {stats.ttftSec}s ·
            {(stats.elapsedMs / 1000).toFixed(1)}s
            {stats.completionTokens > 0 &&
              ` · ${stats.completionTokens} completion-tok`
            }
          </div>
        )}

        {/* Composer */}
        <div className="chat-composer">
          <textarea
            className="chat-input"
            value={input}
            onChange={e => setInput(e.target.value)}
            onKeyDown={e => {
              if (e.key === 'Enter' && (e.ctrlKey || e.metaKey)) {
                e.preventDefault()
                sendMessage()
              }
            }}
            placeholder={
              serverStatus !== 'ok'
                ? '⚠️ Server nicht erreichbar'
                : 'Nachricht eingeben… (Strg+Enter)'
            }
            rows={3}
            disabled={streaming || serverStatus !== 'ok'}
          />
          <div className="cc-actions">
            {!streaming ? (
              <button
                className="btn btn-primary"
                onClick={sendMessage}
                disabled={!input.trim() || serverStatus !== 'ok'}
              >
                ▶ Senden
              </button>
            ) : (
              <button
                className="btn btn-danger"
                onClick={stopStreaming}
              >
                ■ Stop
              </button>
            )}
          </div>
```

```
        </div>

      </div>
    </div>
  )
}

// ── Server-Status-Badge ──────────────────────────────────
function ServerStatusBadge({ status }) {
  const cfg = {
    ok:      { cls: 'badge-ok',      text: '🟢 Bereit' },
    loading: { cls: 'badge-loading', text: '🟡 Lädt…'  },
    error:   { cls: 'badge-error',   text: '🔴 Offline' },
    unknown: { cls: 'badge-unknown', text: '⚪ –'       }
  }[status] || { cls: '', text: '⚪ –' }

  return <span className={`server-badge ${cfg.cls}`}>{cfg.text}</span>
}

// ── Nachricht-Bubble ─────────────────────────────────────
function MessageBubble({ msg }) {
  const isUser = msg.role === 'user'
  return (
    <div className={`msg-bubble ${isUser ? 'user' : 'assistant'} ${msg.error ? 'error' : ''}`}>
      <div className="mb-avatar">{isUser ? '👤' : '🧠'}</div>
      <div className="mb-body">
        <div className="mb-meta">
          <span className="mb-role">{isUser ? 'Du' : 'Hermes'}</span>
          <span className="mb-time">
            {new Date(msg.ts).toLocaleTimeString('de-DE')}
          </span>
        </div>
        <div
          className="mb-content"
          dangerouslySetInnerHTML={{ __html: miniMd(msg.content) }}
        />
        {msg.streaming && (
          <span className="stream-cursor">▋</span>
        )}
        {msg.stats && !msg.streaming && (
          <div className="mb-stats">
            ⚡ {msg.stats.tokens} tok ·
            {msg.stats.tps} tok/s ·
            {(msg.stats.elapsedMs / 1000).toFixed(1)}s
          </div>
        )}
      </div>
    </div>
  )
}

// ── Empty State ──────────────────────────────────────────
function EmptyState({ serverStatus, onSuggestion }) {
  const suggestions = [
    { emoji: '🤖', text: 'Erkläre Transformer-Modelle kurz.' },
    { emoji: '🌀', text: 'Schreibe einen Python HTTP-Server.' },
    { emoji: '🦙', text: 'Was ist der Unterschied: llama.cpp vs Ollama?' },
```

```
    { emoji: '💡', text: '5 Tipps für lokale KI unter Windows.' }
  ]

  if (serverStatus === 'error') {
    return (
      <div className="chat-empty">
        <div className="ce-icon">⚠️</div>
        <h3>Server nicht erreichbar</h3>
        <p>Starte den llama-server auf der Dashboard-Seite.</p>
      </div>
    )
  }

  return (
    <div className="chat-empty">
      <div className="ce-icon">💬</div>
      <h3>Bereit zum Chatten</h3>
      <p>Stelle eine Frage um zu beginnen.</p>
      <div className="ce-suggestions">
        {suggestions.map((s, i) => (
          <button
            key={i}
            className="ce-suggestion"
            onClick={() => onSuggestion(s.text)}
          >
            {s.emoji} {s.text}
          </button>
        ))}
      </div>
    </div>
  )
}

// ── Minimal-Markdown (sicher, kein dangerouslySetInnerHTML-Risiko
//    da input von LLM kommt und nicht vom User direkt) ──────────
function miniMd(text) {
  if (!text) return ''
  let h = text
    .replace(/&/g,'&amp;').replace(/</g,'&lt;').replace(/>/g,'&gt;')

  // Code-Blöcke
  h = h.replace(/```(\w*)\n?([\s\S]*?)```/g,
    (_, lang, code) =>
      `<pre class="code-block"><code>${code.trim()}</code></pre>`
  )
  // Inline-Code
  h = h.replace(/`([^`\n]+)`/g, '<code class="icode">$1</code>')
  // Bold
  h = h.replace(/\*\*([^*\n]+)\*\*/g, '<strong>$1</strong>')
  // Italic
  h = h.replace(/(?<!\*)\*([^*\n]+)\*(?!\*)/g, '<em>$1</em>')
  // Headings
  h = h.replace(/^### (.+)$/gm, '<h4>$1</h4>')
  h = h.replace(/^## (.+)$/gm,  '<h3>$1</h3>')
  h = h.replace(/^# (.+)$/gm,   '<h2>$1</h2>')
  // Listen
  h = h.replace(/^[-*] (.+)$/gm, '<li>$1</li>')
```

```
  h = h.replace(/((?:<li>[\s\S]*?<\/li>)+)/g, '<ul>$1</ul>')
  // HR
  h = h.replace(/^---$/gm, '<hr>')
  // Zeilenumbrüche (außerhalb von Block-Tags)
  h = h.replace(/\n/g, '<br>')
  h = h.replace(/<\/pre><br>/g, '</pre>')
  h = h.replace(/<\/ul><br>/g,  '</ul>')
  h = h.replace(/<hr><br>/g,    '<hr>')

  return h
}

function relTime(ts) {
  const s = Math.floor((Date.now() - ts) / 1000)
  if (s < 60)    return 'gerade eben'
  if (s < 3600)  return `${Math.floor(s/60)}min`
  if (s < 86400) return `${Math.floor(s/3600)}h`
  return new Date(ts).toLocaleDateString('de-DE')
}
```

1️⃣1️⃣ `client/src/App.jsx` — Alles zusammenführen

{} React

```
// ═══════════════════════════════════════════════════════
//  App.jsx — ERGÄNZUNG
//  Füge diese Imports + Komponenten in dein bestehendes App.jsx ein
// ═══════════════════════════════════════════════════════

// Neue Imports:
import { UpdateBanner }  from './components/UpdateBanner'
import { DownloadBar }   from './components/DownloadBar'
import { useDownloader } from './hooks/useDownloader'
import { DownloadsPage } from './pages/DownloadsPage'

// Im App-Komponenten-Body ergänzen:
function App() {
  const {
    activeDownloads,
    cancelDownload
  } = useDownloader()

  // ... dein bestehender App-Code ...

  return (
    <div className="app-root">
      {/* ← Oben: Update-Banner (automatisch nur bei verfügbarem Update) */}
      <UpdateBanner />

      {/* ... dein bestehendes Layout ... */}
```

```
      {/* ← Unten rechts: Download-Bar (automatisch bei aktiven Downloads) */}
      <DownloadBar
        activeDownloads={activeDownloads}
        onCancel={cancelDownload}
      />
    </div>
  )
}
```

1 2 client/src/styles.css — Ergänzungen

{} CSS

```
/* ═══════════════════════════════════════════
   CHAT-SEITE (React-Version)
═══════════════════════════════════════════ */

.chat-root {
  display: grid;
  grid-template-columns: 200px 1fr;
  height: calc(100vh - 48px);
  overflow: hidden;
}

/* Konversations-Sidebar */
.conv-sidebar {
  background: var(--surface);
  border-right: 1px solid var(--border);
  display: flex;
  flex-direction: column;
  overflow: hidden;
}
.conv-top {
  padding: 10px 8px;
  border-bottom: 1px solid var(--border);
  flex-shrink: 0;
}
.conv-list {
  flex: 1;
  overflow-y: auto;
  padding: 4px;
}
.conv-empty {
  padding: 16px 8px;
  font-size: 0.78rem;
  color: var(--text-dim);
  text-align: center;
}
.conv-item {
```

```
  display: flex;
  align-items: center;
  gap: 6px;
  padding: 7px 8px;
  border-radius: 6px;
  cursor: pointer;
  transition: background 0.12s;
  margin-bottom: 2px;
}
.conv-item:hover   { background: var(--surface2); }
.conv-item.active  {
  background: rgba(124, 58, 237, 0.12);
  border-left: 3px solid var(--accent);
  padding-left: 5px;
}
.ci-body  { flex: 1; min-width: 0; }
.ci-title {
  font-size: 0.8rem;
  font-weight: 600;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
.ci-meta  { font-size: 0.65rem; color: var(--text-dim); margin-top: 2px; }
.ci-del   {
  opacity: 0;
  background: transparent;
  border: none;
  color: var(--text-dim);
  cursor: pointer;
  padding: 2px 4px;
  border-radius: 3px;
  font-size: 0.7rem;
  flex-shrink: 0;
  transition: opacity 0.1s;
}
.conv-item:hover .ci-del { opacity: 1; }
.ci-del:hover { color: var(--red); }

/* Chat-Main */
.chat-main {
  display: flex;
  flex-direction: column;
  overflow: hidden;
  background: var(--bg);
}
.chat-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 8px 14px;
  border-bottom: 1px solid var(--border);
  background: var(--surface);
  flex-shrink: 0;
  gap: 10px;
}
.ch-left, .ch-right { display: flex; align-items: center; gap: 8px; }
```

```
.ch-model { font-size: 0.78rem; color: var(--text-dim); font-family: var(--font-mono); }

/* Server-Badge */
.server-badge {
  font-size: 0.78rem;
  padding: 3px 8px;
  border-radius: 20px;
  border: 1px solid var(--border);
}
.badge-ok      { color: var(--green);   border-color: rgba(34,197,94,0.3); }
.badge-loading { color: var(--yellow);  border-color: rgba(234,179,8,0.3); }
.badge-error   { color: var(--red);     border-color: rgba(239,68,68,0.3); }
.badge-unknown { color: var(--text-dim);}

/* Chat-Optionen */
.chat-options {
  background: var(--surface2);
  border-bottom: 1px solid var(--border);
  flex-shrink: 0;
}
.chat-options summary {
  padding: 6px 14px;
  cursor: pointer;
  font-size: 0.8rem;
  color: var(--text-dim);
  list-style: none;
}
.co-grid  { padding: 10px 14px; display: flex; flex-direction: column; gap: 8px; }
.co-field { display: flex; flex-direction: column; gap: 3px; }
.co-field label { font-size: 0.72rem; color: var(--text-dim); }
.co-field textarea, .co-field input[type=number] {
  background: var(--surface);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 5px 8px;
  border-radius: 6px;
  font-family: inherit;
  font-size: 0.82rem;
  resize: vertical;
}
.co-row   { display: flex; gap: 12px; }
.co-field input[type=range] { accent-color: var(--accent); }

/* Nachrichten */
.chat-messages {
  flex: 1;
  overflow-y: auto;
  padding: 14px;
  display: flex;
  flex-direction: column;
  gap: 10px;
}

/* Empty State */
.chat-empty {
  text-align: center;
  padding: 40px 20px;
```

```
  color: var(--text-dim);
  margin: auto;
}
.ce-icon  { font-size: 2.5rem; margin-bottom: 10px; }
.chat-empty h3 { color: var(--text); margin-bottom: 6px; }
.ce-suggestions {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  justify-content: center;
  margin-top: 16px;
}
.ce-suggestion {
  background: var(--surface);
  border: 1px solid var(--border);
  color: var(--text-dim);
  padding: 7px 12px;
  border-radius: 20px;
  cursor: pointer;
  font-size: 0.78rem;
  transition: all 0.12s;
}
.ce-suggestion:hover { border-color: var(--accent); color: var(--text); }

/* Bubbles */
.msg-bubble {
  display: flex;
  gap: 8px;
  max-width: 88%;
  animation: mbIn 0.12s ease;
}
@keyframes mbIn {
  from { opacity:0; transform:translateY(4px); }
  to   { opacity:1; transform:translateY(0); }
}
.msg-bubble.user      { align-self: flex-end; flex-direction: row-reverse; }
.msg-bubble.assistant { align-self: flex-start; }

.mb-avatar {
  width: 28px; height: 28px;
  border-radius: 50%;
  background: var(--surface2);
  border: 1px solid var(--border);
  display: flex; align-items: center; justify-content: center;
  font-size: 0.85rem;
  flex-shrink: 0;
  margin-top: 2px;
}
.msg-bubble.assistant .mb-avatar {
  background: rgba(124,58,237,0.1);
  border-color: rgba(124,58,237,0.3);
}

.mb-body { flex: 1; min-width: 0; }
.mb-meta { display: flex; gap: 8px; align-items: baseline; margin-bottom: 4px; }
.mb-role { font-size: 0.75rem; font-weight: 700; }
.mb-time { font-size: 0.65rem; color: var(--text-dim); }
```

```css
.mb-content {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 10px 10px 10px 3px;
  padding: 9px 12px;
  font-size: 0.85rem;
  line-height: 1.65;
  word-break: break-word;
}
.msg-bubble.user .mb-content {
  background: rgba(124,58,237,0.08);
  border-color: rgba(124,58,237,0.25);
  border-radius: 10px 10px 3px 10px;
}
.msg-bubble.error .mb-content {
  background: rgba(239,68,68,0.06);
  border-color: rgba(239,68,68,0.3);
}

/* Markdown in Bubbles */
.mb-content .code-block {
  background: var(--bg);
  border: 1px solid var(--border);
  border-radius: 6px;
  padding: 8px 10px;
  overflow-x: auto;
  font-size: 0.78rem;
  font-family: var(--font-mono);
  margin: 6px 0;
}
.mb-content .icode {
  background: rgba(124,58,237,0.1);
  padding: 1px 4px;
  border-radius: 3px;
  font-size: 0.8rem;
  font-family: var(--font-mono);
}
.mb-content ul { padding-left: 16px; margin: 4px 0; }

.mb-stats {
  font-size: 0.65rem;
  color: var(--text-dim);
  font-family: var(--font-mono);
  margin-top: 4px;
}

.stream-cursor {
  animation: scblink 0.9s step-end infinite;
  color: var(--accent);
  font-weight: 700;
}
@keyframes scblink { 0%,49%{opacity:1} 50%,100%{opacity:0} }

/* Chat Stats */
.chat-stats-bar {
  padding: 4px 14px;
```

```
  font-size: 0.72rem;
  color: var(--accent);
  font-family: var(--font-mono);
  border-top: 1px solid var(--border);
  flex-shrink: 0;
}

/* Composer */
.chat-composer {
  display: flex;
  gap: 8px;
  padding: 10px 12px;
  background: var(--surface);
  border-top: 1px solid var(--border);
  flex-shrink: 0;
}
.chat-input {
  flex: 1;
  background: var(--surface2);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 8px 10px;
  border-radius: 8px;
  font-family: inherit;
  font-size: 0.86rem;
  resize: none;
  line-height: 1.5;
}
.chat-input:focus { outline: none; border-color: var(--accent); }
.cc-actions { display: flex; flex-direction: column; justify-content: center; }

/* ═══════════════════════════════════════════
   UPDATE-BANNER
═══════════════════════════════════════════ */

.update-banner {
  position: fixed;
  top: 50px;
  right: 14px;
  z-index: 2000;
  background: var(--surface);
  border: 1px solid var(--accent);
  border-radius: 10px;
  min-width: 340px;
  max-width: 460px;
  box-shadow: 0 8px 28px rgba(0,0,0,0.45);
  animation: ubSlide 0.2s ease;
  overflow: hidden;
}
@keyframes ubSlide {
  from { opacity:0; transform:translateX(14px); }
  to   { opacity:1; transform:translateX(0); }
}
.ub-error    { border-color: var(--red); }
.ub-downloaded {
  border-color: var(--green);
  box-shadow: 0 0 20px rgba(34,197,94,0.2);
```

```
}

.ub-content {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 12px 14px;
  flex-wrap: wrap;
}
.ub-texts  { flex: 1; min-width: 160px; }
.ub-title  { display: block; font-size: 0.88rem; font-weight: 600; margin-bottom: 2px; }
.ub-sub    { font-size: 0.75rem; color: var(--text-dim); }
.ub-actions { display: flex; gap: 6px; }

.ub-progress-wrap { width: 100%; padding: 0 14px 10px; }
.ub-progress-bar  {
  height: 5px;
  background: var(--surface2);
  border-radius: 3px;
  overflow: hidden;
}
.ub-progress-fill {
  height: 100%;
  background: var(--accent);
  border-radius: 3px;
  transition: width 0.25s;
}

/* ═══════════════════════════════════════════
   DOWNLOAD-BAR (Floating)
═══════════════════════════════════════════ */

.dl-float-bar {
  position: fixed;
  bottom: 14px;
  right: 14px;
  z-index: 900;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 10px;
  width: 360px;
  max-width: calc(100vw - 28px);
  box-shadow: 0 6px 20px rgba(0,0,0,0.4);
  overflow: hidden;
}
.dlb-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 8px 12px;
  background: var(--surface2);
  border-bottom: 1px solid var(--border);
}
.dlb-title { font-size: 0.82rem; font-weight: 600; }
.dlb-list  { max-height: 240px; overflow-y: auto; padding: 6px; }

.dlb-item {
```

```
  background: var(--bg);
  border: 1px solid var(--border);
  border-radius: 7px;
  padding: 8px 10px;
  margin-bottom: 4px;
}
.dlb-item:last-child { margin-bottom: 0; }
.dlb-row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 4px;
}
.dlb-name {
  font-size: 0.78rem;
  font-weight: 600;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  max-width: 270px;
}
.dlb-stats {
  display: flex;
  gap: 8px;
  font-size: 0.67rem;
  color: var(--text-dim);
  font-family: var(--font-mono);
  margin-bottom: 5px;
}
.dlb-pct   { margin-left: auto; color: var(--accent); }
.dlb-bar-wrap { }
.dlb-bar   { height: 5px; background: var(--surface2); border-radius: 3px; overflow: hidden; }
.dlb-fill  { height: 100%; background: var(--accent); border-radius: 3px; transition: width 0.2s; }
.dlb-done  { font-size: 0.72rem; color: var(--green); margin-top: 4px; }
.dlb-error { font-size: 0.72rem; color: var(--red);   margin-top: 4px; }

/* ═══════════════════════════════════════════
   DOWNLOADS-PAGE
═══════════════════════════════════════════ */

.tab-bar {
  display: flex;
  gap: 2px;
  border-bottom: 1px solid var(--border);
  margin-bottom: 16px;
}
.tab-btn {
  padding: 8px 16px;
  background: transparent;
  border: none;
  border-bottom: 2px solid transparent;
  color: var(--text-dim);
  cursor: pointer;
  font-size: 0.85rem;
  transition: all 0.15s;
  display: flex;
  align-items: center;
```

```
  gap: 6px;
}
.tab-btn:hover  { color: var(--text); }
.tab-btn.active {
  color: var(--accent);
  border-bottom-color: var(--accent);
}
.tab-badge {
  background: var(--accent);
  color: white;
  border-radius: 10px;
  padding: 1px 6px;
  font-size: 0.7rem;
  font-weight: 700;
}
.tab-count { color: var(--text-dim); font-size: 0.72rem; }

.tab-content { }

.dl-active-list { display: flex; flex-direction: column; gap: 10px; }
.dl-history-toolbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 12px;
}
.dl-count { font-size: 0.78rem; color: var(--text-dim); }
.dl-history-list { display: flex; flex-direction: column; gap: 8px; }

/* Download-Karte */
.dl-card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 14px;
}
.dlc-header {
  display: flex;
  align-items: flex-start;
  gap: 12px;
  margin-bottom: 10px;
}
.dlc-icon { font-size: 1.4rem; flex-shrink: 0; }
.dlc-info { flex: 1; }
.dlc-name {
  font-weight: 700;
  font-size: 0.9rem;
  word-break: break-all;
  margin-bottom: 4px;
}
.dlc-stats {
  display: flex;
  gap: 10px;
  font-size: 0.75rem;
  color: var(--text-dim);
  font-family: var(--font-mono);
}
```

```css
.dlc-progress { display: flex; align-items: center; gap: 8px; }
.dlc-bar  { flex: 1; height: 6px; background: var(--surface2); border-radius: 3px; overflow: hidden; }
.dlc-fill { height: 100%; background: var(--accent); border-radius: 3px; transition: width 0.2s; }
.dlc-pct  { font-size: 0.75rem; font-family: var(--font-mono); color: var(--accent); min-width: 32px; }

/* Verlaufs-Karte */
.dl-history-card {
  display: flex;
  align-items: flex-start;
  gap: 10px;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 10px 12px;
  transition: border-color 0.15s;
}
.dl-history-card:hover { border-color: var(--accent); }
.dhc-icon  { font-size: 1.3rem; flex-shrink: 0; padding-top: 2px; }
.dhc-body  { flex: 1; min-width: 0; }
.dhc-name  { font-weight: 700; font-size: 0.88rem; margin-bottom: 4px; }
.dhc-meta  {
  display: flex;
  gap: 10px;
  font-size: 0.72rem;
  color: var(--text-dim);
  flex-wrap: wrap;
  margin-bottom: 4px;
}
.dhc-path  {
  font-family: var(--font-mono);
  font-size: 0.68rem;
  color: var(--text-dim);
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
.dhc-actions { flex-shrink: 0; }

/* Gemeinsame Empty + Loading States */
.empty-state {
  text-align: center;
  padding: 50px 20px;
  color: var(--text-dim);
}
.es-icon  { font-size: 2.5rem; margin-bottom: 12px; }
.empty-state h3  { color: var(--text); margin-bottom: 8px; }

.loading-state {
  padding: 30px;
  text-align: center;
  color: var(--text-dim);
}
```

`package.json` — Ergänzungen

{} JSON

```json
{
  "dependencies": {
    "electron-store":   "^8.2.0",
    "electron-updater": "^6.3.9"
  },
  "devDependencies": {
    "electron":         "^36.0.0",
    "electron-builder": "^24.13.3"
  },
  "build": {
    "publish": {
      "provider":    "github",
      "owner":       "DEIN_GITHUB_USERNAME",
      "repo":        "hermes-control-center",
      "releaseType": "release"
    }
  }
}
```

Quick-Start

{} Bash

```bash
# Abhängigkeiten
npm install
npm install electron-updater

# Entwickeln
npm run dev           # Electron + React

# Bauen (Auto-Updater nur hier aktiv!)
npm run build

# Release auf GitHub
$env:GH_TOKEN = "ghp_xxx"
npm run release
```

Vollständige React-Architektur v3.0

{} text

```
client/src/
├── hooks/
│   ├── useUpdater.js          ✅ NEU — Auto-Updater State
│   ├── useDownloader.js       ✅ NEU — Download Manager
│   └── useServerStatus.js     ✅ NEU — Health-Polling
├── components/
│   ├── UpdateBanner.jsx       ✅ NEU — Update-Benachrichtigung
│   └── DownloadBar.jsx        ✅ NEU — Floating Progress
├── pages/
│   ├── ChatPage.jsx           ✅ VERBESSERT — SSE + Export
│   ├── DownloadsPage.jsx      ✅ NEU — Verlauf + Manager
│   └── ... (bestehend)
└── App.jsx                    ← UpdateBanner + DownloadBar einbauen
```

Feature	Status
Chat-Streaming SSE	✅
SSE-Bug-Workaround llama.cpp	✅
Timings aus letztem Chunk	✅
Multi-Konversation + Persistenz	✅
Chat-Export .md + .json	✅
🆕 Auto-Updater Hook	✅
🆕 Update-Banner React	✅
🆕 In-App Downloader	✅
🆕 Download-Progress Live	✅
🆕 Download-Verlauf	✅
🆕 Floating Download-Bar	✅
🆕 useServerStatus Hook	✅

> 📌 *Nächste Optionen:*
>
> - `E` — *Code-Signing (kein SmartScreen auf Windows)*
> - `F` — *HF-Modell-Browser mit direktem Download-Button*
> - `G` — *Erweiterte Einstellungen: Theme, Polling-Intervall, Download-Ordner*
> - `H` — `docs/KI_UEBERGABE_V4.md` — *Finaler Übergabe-Prompt für Google AI Studio*